PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

AI-Based Vulnerability Prioritization & Patch Management System

AN INTERNSHIP REPORT

Submitted by

ANIKETH KUMAR – 20221CCS0124

Under the guidance of,

Mr. SAKTHIVEL E

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (Cyber Security)

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "AI-Based Vulnerability Prioritization & Patch Management System" is a bonafide work of ANIKETH KUMAR (20221CCS0124), who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING (SPECIALIZATION DIVISION), CYBER SECURITY during 2025-2026.

Mr. Sakthivel E
Assistant Professor
Internship Guide PSCS
(Spl. Division)
Presidency University

Dr. Geetha A
Associate Professor
Program Internship Coordinator
PSCS (Spl.Division)
Presidency University

Dr. Geetha A
Associate Professor
School Internship Coordinator
PSCS (Spl.Division)
Presidency University

24/4/2026

Dr. Anandaraj S.P
Professor & Head of the Department PSCS
(Spl. Division)
Presidency University

Dr. Shakkeera L
Associate Dean PSCS (Spl. Division)
Presidency University

Sl.No	Name of the Examiner	Signature
1	Ms.Padma T G	Padma T.G
2	Ms.Raesa Razeen	Raesa

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING (Cyber Security), at Presidency University, Bengaluru, named Aniketh Kumar, hereby declare that the internship work titled "AI-Based Vulnerability Prioritization & Patch Management System" has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING (Spl. Division) during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Aniketh Kumar USN: 20221CCS0124 Signature: Aniketh

PLACE: BENGALURU

DATE: 25 April 2026

INTERNSHIP OFFER LETTER

HACKERHUB8 LLP
Built with Purpose. Secured with Passion. Inspired by Innovation
ISO 9001:2015 Certified

+91-9606969233
support@hackerhub8.in
www.hackerhub8.in
Bengaluru, Karnataka

Internship Offer Letter

HackerHub8 LLP |
CIN – ACR-1466

Date: 14 Dec 2025

Dear **Aniketh Kumar**,

We are pleased to offer you an internship position at **HackerHub8 LLP**. This letter confirms your selection for the role of **Internship Role – Jr. Cyber Security Intern** for the duration **from Start** 04 Jan 2026 **to** 01 May 2026.

Internship Details

Role: Jr. Cyber Security Intern
Department: Cybersecurity
Mode: Online
Working Hours: Flexible (Updated on Mail)

Your Tasks & Responsibilities Include:

- Participating in daily/weekly live sessions, labs, and training modules
- Working on assigned projects, tasks, or assessments
- Reporting progress to the allocated mentor
- Following organizational guidelines and maintaining confidentiality at all times

Terms & Conditions

You are expected to conduct yourself professionally throughout the internship period and uphold the values and policies of HackerHub8 LLP . Any form of misconduct or policy violation may lead to immediate termination of the internship.

Certification

Upon successful completion of your internship and assigned tasks, you will receive an **Internship Completion Certificate** from HackerHub8 LLP.
We are excited to welcome you as part of our team and look forward to your valuable contributions.
Kindly confirm your acceptance of this offer to proceed with the onboarding process.

Welcome to HackerHub8 LLP Team
Warm Regards,
HR Manager | Ravi G

HACKERHUB8 LLP SINCE 2019 KARNATAKA

Governed by the laws of India under the **" Information Technology Act, 2000 "**

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved Chancellor, Pro-Vice Chancellor, and Registrar for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide Mr. Sakthivel E, Assistant Professor, Presidency School of Computer Science and Engineering (Spl. Division), Presidency University, for his moral support, motivation, timely guidance, and encouragement provided to me during the period of internship work.

I am also thankful to Dr. Anandaraj S.P, Professor, Head of the Department, Presidency School of Computer Science and Engineering (Spl. Division), Presidency University, for his mentorship and encouragement.

I express my cordial thanks to Dr. Shakkeera L, Associate Dean, Presidency School of Computer Science and Engineering (Spl. Division), Presidency University, for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to Dr Geetha A**,** School Internship Coordinator and Program Internship Coordinator, Presidency School of Information Science and Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to the Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

I extend my sincere gratitude to HackerHub8 LLP, Bengaluru, for providing me with the opportunity to work as a Jr. Cyber Security Intern and for their continuous guidance and support throughout the internship period.

Aniketh Kumar

ABSTRACT

The exponential growth in software vulnerabilities has created a critical challenge for cybersecurity teams: identifying which vulnerabilities to fix first. Traditional tools such as Nessus and OpenVAS flood security teams with hundreds of unranked CVEs weekly, relying solely on CVSS scores to prioritize remediation. However, CVSS alone is insufficient as it ignores contextual factors such as exploit availability, asset criticality, internet exposure, and active threat intelligence.

This internship project focuses on the design and development of an AI-Based Vulnerability Prioritization and Patch Management System at HackerHub8 LLP. The system replaces simple CVSS-based ranking with a 5-factor AI-weighted priority score ranging from 0 to 100. The five factors are: CVSS Base Score (30%), Exploit Availability (20%), Asset Criticality (20%), Internet Exposure (15%), and Threat Intelligence Match (15%). Each vulnerability is automatically classified into one of four action tiers: Immediate Patch (80–100), High Priority (60–79), Scheduled (40–59), and Monitor (below 40).

The system was built using a modern full-stack architecture comprising Python 3.10 with FastAPI as the backend, React 19 as the frontend, PostgreSQL 15 as the database, and Docker for containerized deployment. The backend exposes 13 REST API endpoints and manages 10 interrelated database tables. The frontend provides a 4-tab dashboard displaying KPI metrics, risk trend charts, patch management workflows, and department-wise exposure analytics.

The system was evaluated using a dataset of 36 vulnerabilities across 11 departments. The results demonstrated that the AI scoring engine accurately differentiates high-risk vulnerabilities from low-risk ones, enabling security teams to prioritize remediation effectively. All 13 automated API tests passed, confirming the correctness of every system module. Overall, the hybrid multi-factor approach provides a more robust and actionable vulnerability management framework compared to conventional CVSS-only systems.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	i
	Acknowledgement	iii
	Abstract	iv
	List of Figures	viii
	List of Tables	ix
	Abbreviations	x
1.	Introduction 1.1 Background 1.2 Statistics of Internship 1.3 Prior Existing Technologies 1.4 Proposed Approach 1.5 Objectives 1.6 SDGs 1.7 Overview of Report	01
2.	Literature Survey	06
3.	System Architecture & Technology Stack	09
4.	Project Management 4.1 Internship Timeline 4.2 Risk Analysis 4.3 Internship Budget	12

	4.4 Gantt Chart Explanation 4.5 Resource Allocation 4.6 Risk Mitigation Strategies	
5.	System Design & Implementation 5.1 AI Scoring Engine 5.2 Database Design 5.3 API Endpoints 5.4 Frontend Dashboard 5.5 AI Scoring Engine Logic (Detailed) 5.6 Patch Lifecycle Flow 5.7 Authentication System 5.8 Error Handling and Logging	15
6.	Hardware, Software and Simulation 6.1 Hardware Requirements 6.2 Software Tools and Development Environment 6.3 Simulation and Experimental Setup 6.4 System Performance Benchmarks 6.5 System Limitations	22
7.	Evaluation and Results 7.1 Introduction 7.2 Results and Observations 7.3 Performance Analysis 7.4 Case Study Analysis	25
8.	Social, Legal and Ethical Aspects 8.1 Introduction 8.2 Social Impact 8.3 Legal Considerations 8.4 Ethical Considerations	31

9.	Learning Outcomes	35
10.	Conclusion and Future Scope	39
11	References	49

LIST OF FIGURES

Sl. No.	Figure No.	Title	Page No.
1	Figure 1.1	AI Vulnerability Prioritization System Architecture	02
2	Figure 3.1	3-Tier Web Architecture Diagram	09
3	Figure 3.2	End-to-End Data Flow Diagram	11
4	Figure 5.1	AI Scoring Engine — Weighted Formula	16
5	Figure 5.2	Classification Tiers — Priority Score Ranges	16
6	Figure 5.3	Database Schema — 10 Table ERD	18
7	Figure 5.4	Vulnerability Ingestion Pipeline (_ingest_one())	19
8	Figure 5.5	Tab 1 — Dashboard with KPIs, Risk Trend Chart and Vulnerability Table	20
9	Figure 5.6	Tab 2 — Patch Management with AI Score and Inline Status Editing	20
10	Figure 5.7	Tab 3 — Upload Scan with CSV Drag-and-Drop and JSON Payload Input	21
11	Figure 5.8	Tab 4 — Department Exposure Overview with Bar Chart and Risk Level Table	21
12	Figure 6.1	Docker Compose Architecture	23
13	Figure 7.1	Live Dashboard — 36 Vulnerabilities, 10 Critical, Risk Trend over 36 Scans	29
14	Figure 7.2	Department Exposure — Infrastructure Highest Risk (9 Vulns, 8 Critical)	30

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 1.1	Statistics of the Internship	02
2	Table 1.2	Prior Existing Vulnerability Management Systems	03
3	Table 1.3	SDGs and Project Relevance	05
4	Table 3.1	Technology Stack Summary	10
5	Table 4.1	Internship Timeline	12
6	Table 4.2	Risk Analysis	13
7	Table 4.3	Budget Plan	14
8	Table 5.1	AI Scoring Factors and Weights	15
9	Table 5.2	Database Tables and Their Purpose	17
10	Table 5.3	API Endpoints — All 13 Routes	18
11	Table 6.1	Hardware Configuration	22
12	Table 6.2	Software Tools and Their Purpose	23
13	Table 7.1	Sample Vulnerability Dataset	25
14	Table 7.2	Test Results — All 13 API Tests Passing	26

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DB	Database
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JWT	JSON Web Token
KPI	Key Performance Indicator
MVP	Minimum Viable Product
NVD	National Vulnerability Database
ORM	Object Relational Mapper
RAM	Random Access Memory
REST	Representational State Transfer
SDG	Sustainable Development Goal
SIEM	Security Information and Event Management
SSD	Solid State Drive
SQL	Structured Query Language
UI	User Interface
VAPT	Vulnerability Assessment and Penetration Testing
WAF	Web Application Firewall

Chapter 1

INTRODUCTION

1.1 Background

The rapid expansion of digital infrastructure across enterprises and organizations has led to an unprecedented increase in software vulnerabilities. Every week, security teams are flooded with hundreds of Common Vulnerabilities and Exposures (CVEs) generated by scanning tools such as Nessus and OpenVAS. While these tools are effective at identifying vulnerabilities, they lack the intelligence to determine which vulnerabilities pose the greatest real-world risk.

Traditional vulnerability management relies heavily on the CVSS (Common Vulnerability Scoring System) score, a standardized severity rating from 0 to 10. While CVSS provides a useful baseline, it fails to account for critical contextual factors. A vulnerability rated CVSS 9.8 on an internal test server may pose less risk than a CVSS 7.2 vulnerability on an internet-facing payment gateway. Similarly, if a known exploit is publicly available for a particular CVE, its urgency increases dramatically regardless of its CVSS score.

According to industry reports, over 60% of security breaches exploit known vulnerabilities for which patches were available but not yet applied. The root cause is not a lack of awareness but an inability to efficiently decide which vulnerabilities to fix first given limited security resources. This project addresses this challenge by developing an AI-Based Vulnerability Prioritization and Patch Management System at HackerHub8 LLP.

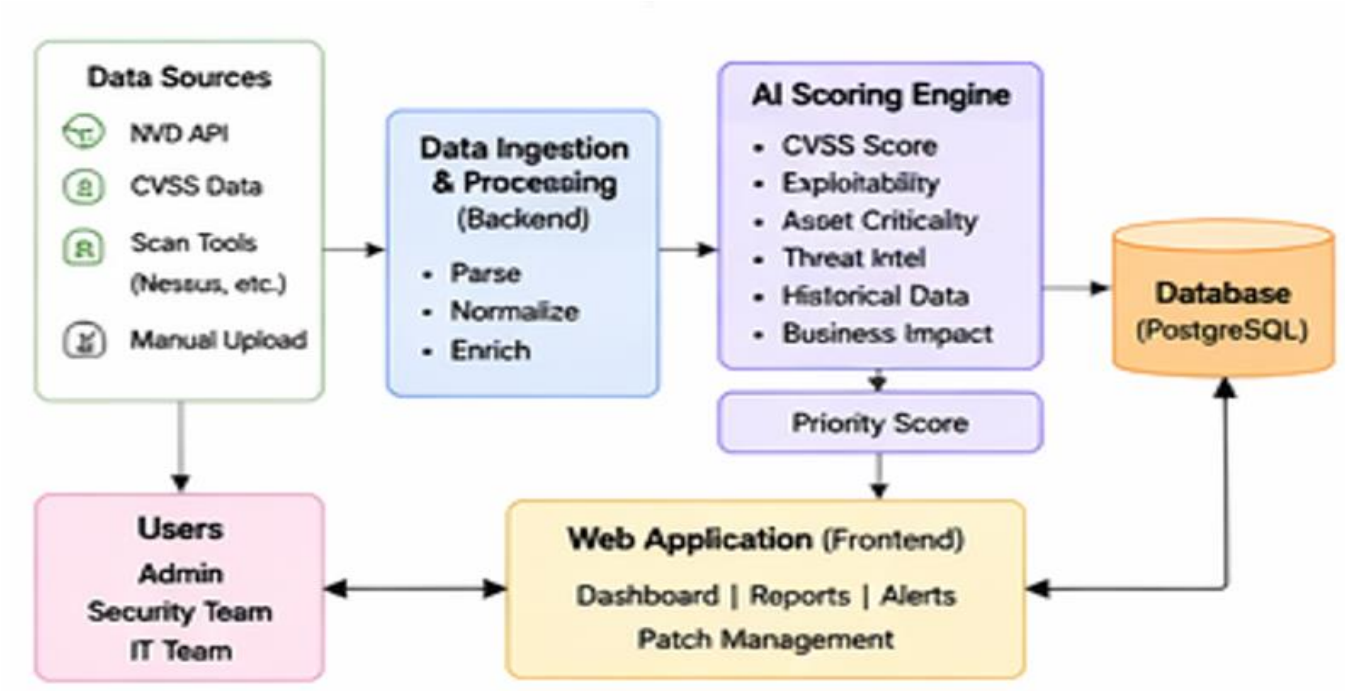


Figure 1.1:AI Vulnerability Prioritization System Architecture

1.2 Statistics of Internship

The internship was conducted at HackerHub8 LLP, Bengaluru, Karnataka, an ISO 9001:2015 certified cybersecurity firm, in online mode as a Jr. Cyber Security Intern for approximately four months. The primary objective was to design, develop, and document a complete AI-driven vulnerability management platform.

Table 1.1: Statistics of the Internship

Parameter	Details
Internship Title	AI-Based Vulnerability Prioritization & Patch Management System
Organization	HackerHub8 LLP, Bengaluru, Karnataka
Department	Cybersecurity
Mode	Online

Duration	04 January 2026 to 01 May 2026
Domain	Cybersecurity, AI/ML, Full-Stack Web Development
Primary Tech Stack	Python, FastAPI, React, PostgreSQL, Docker
Main Output	Web-based AI Vulnerability Management Platform — 13 API endpoints, 10 DB tables

1.3 Prior Existing Technologies

Vulnerability management has evolved considerably over the past decade. Several tools and frameworks exist for identifying and managing software vulnerabilities, but most rely on static scoring and lack intelligent prioritization.

Table 1.2: Prior Existing Vulnerability Management Systems

Category	Systems/Tools	Strength
Traditional Scanners	Nessus, OpenVAS, Qualys	Comprehensive vulnerability discovery
CVSS-Based Prioritization	NVD Database, CVE Details	Standardized severity scoring
SIEM Platforms	Splunk, IBM QRadar	Event correlation and monitoring
Patch Management	WSUS, SCCM, ManageEngine	Automated patch deployment
Threat Intelligence	VirusTotal, MISP, OpenCTI	Active threat actor tracking
Integrated Platforms	Tenable.io, Rapid7 InsightVM	Unified vulnerability lifecycle management

1.4 Proposed Approach

The proposed system is an AI-Based Vulnerability Prioritization and Patch Management Platform that integrates vulnerability data, threat intelligence, and asset criticality into a multi-factor scoring engine. The core innovation is replacing CVSS-only ranking with a 5-factor AI-weighted priority score (0–100). The formula is:

Priority Score = (CVSS × 10 × 0.30) + (Exploit × 100 × 0.20) + (Asset Criticality × 0.20) + (Internet Exposure × 0.15) + (Threat Intel × 0.15)

The resulting score classifies each vulnerability into: Immediate Patch (80–100), High Priority (60–79), Scheduled (40–59), or Monitor (below 40).

1.5 Objectives

- Study the limitations of traditional CVSS-based vulnerability prioritization and identify the need for a multi-factor intelligent scoring system.
- Design and implement a 5-factor AI scoring engine that produces a dynamic priority score based on real-world risk context.
- Develop a full-stack web application using Python FastAPI, React, and PostgreSQL for vulnerability ingestion, prioritization, and patch tracking.
- Build a comprehensive patch management module with status lifecycle tracking, owner assignment, audit history, and compliance reporting.
- Evaluate the system using sample vulnerability datasets and validate the scoring engine against expected classification outcomes.

- Apply secure software development principles including JWT authentication, bcrypt password hashing, and CORS configuration.

1.6 SDGs (Sustainable Development Goals)

Table 1.3: SDGs and Project Relevance

SDG	Relevance to the Project
SDG 4 - Quality Education	The internship enhanced technical skills in AI, cybersecurity, and full-stack development.
SDG 9 - Industry, Innovation and Infrastructure	The system contributes to secure digital infrastructure through proactive vulnerability management.
SDG 16 - Peace, Justice and Strong Institutions	Automated vulnerability management reduces cybercrime, digital fraud, and data breaches.
SDG 17 - Partnerships for the Goals	The work benefits from industry mentorship at HackerHub8 LLP and Presidency University guidance.

1.7 Overview of the Report

This report is structured to present the internship work systematically. Chapter 1 introduces the background, statistics, prior technologies, proposed approach, objectives, and SDG alignment. Chapter 2 presents a detailed literature survey. Chapter 3 describes the system architecture and technology stack. Chapter 4 covers project management. Chapters 5 through 9 present system design, hardware/software, evaluation, social aspects, and learning outcomes respectively. Chapter 10 concludes with future scope.

Chapter 2

LITERATURE SURVEY

2.1 Introduction

Vulnerability management has been an active area of research in cybersecurity since the early 2000s when the CVE system was introduced by MITRE Corporation. Over time, the field has evolved from simple patch scheduling to sophisticated risk-based approaches that consider multiple contextual factors. This chapter reviews the key developments in vulnerability management, AI-based risk scoring, and patch prioritization that form the foundation for this project.

2.2 Traditional CVSS-Based Vulnerability Management

The Common Vulnerability Scoring System (CVSS) was introduced as a standardized framework for rating the severity of security vulnerabilities. CVSS v3.1 scores vulnerabilities on a scale of 0 to 10 based on factors such as attack vector, attack complexity, privileges required, user interaction, and impact on confidentiality, integrity, and availability.

While CVSS provides a universally accepted severity baseline, research consistently demonstrates its limitations for prioritization purposes. Studies show that fewer than 10% of published CVEs are ever exploited in the wild, yet organizations attempt to patch all high-severity CVEs, resulting in resource exhaustion and patch fatigue. The CVSS score does not account for whether an exploit is publicly available, whether the affected asset is internet-facing, or whether the vulnerability is being actively targeted by threat actors.

2.3 Exploit Availability and Real-World Risk

Several researchers have studied the relationship between CVSS scores and actual exploitation rates. Sabetta and Bezzi (2018) found that only about 1.5% of publicly known vulnerabilities are actually exploited in practice. This finding highlights the critical importance of using additional signals beyond CVSS to prioritize remediation efforts.

The EPSS (Exploit Prediction Scoring System), introduced by FIRST.org, attempts to address this by predicting the probability of exploitation within 30 days based on multiple threat intelligence signals. This

represents a significant step toward more contextual vulnerability scoring, an approach that this project builds upon.

2.4 Asset Criticality-Based Prioritization

Asset criticality is a key dimension of vulnerability risk that traditional CVSS scoring ignores entirely. A critical vulnerability on a non-production internal test server presents fundamentally different risk from the same vulnerability on a payment gateway or healthcare records system. Risk-based vulnerability management (RBVM) frameworks, popularized by vendors such as Tenable and Rapid7, incorporate asset criticality into their scoring methodologies.

Research by Jacobs et al. (2020) demonstrated that combining CVSS scores with asset context and threat intelligence data significantly improved prioritization accuracy. Their work showed that contextual scoring could reduce the number of patches needed to address 50% of actively exploited vulnerabilities by a factor of four compared to pure CVSS-based prioritization.

2.5 Machine Learning in Vulnerability Prioritization

Recent years have seen increasing interest in applying machine learning to vulnerability management. Bozorgi et al. (2010) pioneered the use of supervised machine learning to predict whether a vulnerability would be exploited, using features extracted from CVE descriptions and metadata. More recent work has explored BERT-based NLP models to extract structured information from unstructured CVE text, enabling more nuanced risk assessment.

2.6 Patch Management and Compliance

Patch management as a discipline focuses on the systematic process of acquiring, testing, and deploying patches to software systems. Industry frameworks such as the NIST Cybersecurity Framework and CIS Controls emphasize timely patching as a foundational security control. CIS Control 7 specifically recommends a risk-based patch prioritization process that considers asset criticality and exploit availability.

2.7 Threat Intelligence Integration

Threat intelligence refers to information about current and emerging threats, including knowledge of which vulnerabilities are being actively exploited by specific threat actors. Platforms such as MISP and

OpenCTI enable organizations to consume structured threat intelligence. Research has shown that vulnerabilities matched against active threat intelligence feeds should receive immediate attention regardless of their CVSS score.

2.8 Research Gap

The review of existing literature reveals that while individual components of intelligent vulnerability management have been studied extensively, few systems bring all these components together into a unified, accessible, and automated platform. Most existing commercial solutions are expensive enterprise tools inaccessible to small and medium enterprises. This project addresses this gap by developing a complete end-to-end vulnerability management platform that combines AI scoring, patch lifecycle management, and risk trend analytics.

Chapter 3

SYSTEM ARCHITECTURE & TECHNOLOGY STACK

3.1 System Architecture Overview

The system follows a classic 3-tier web architecture where a React frontend communicates with a FastAPI backend through HTTP REST APIs, and the backend persists all data to a PostgreSQL database. Everything is containerized with Docker for easy and reproducible deployment. The architecture is divided into six logical layers: Presentation (React on port 3000), API (FastAPI on port 8000), Vulnerability Processing Engine, AI Prioritization Engine, Patch Management Module, and Data Storage (PostgreSQL on port 5432).

A key design choice is optional JWT authentication. All data endpoints function without a token for demo and development purposes, while users can register and obtain a bearer token for protected production operations. This ensures the system is fully testable without authentication overhead.

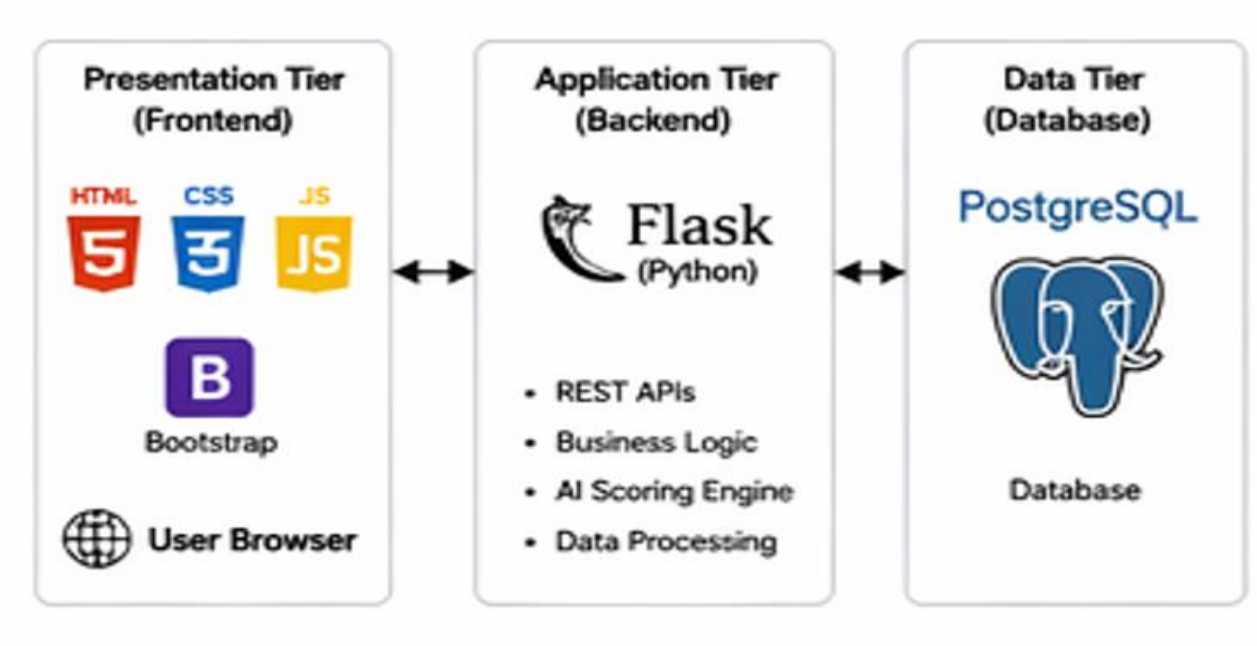


Figure 3.1: 3-Tier Web Architecture Diagram

3.2 Data Flow

The end-to-end data flow begins when a user uploads a CSV file through the Upload Scan tab. React sends a POST request to /api/v1/ingest-csv. FastAPI parses it using Python's csv.DictReader, validates columns via Pydantic, and calls the _ingest_one() pipeline for each row. This 7-step pipeline: creates the department, creates the asset, stores the CVE, creates the vulnerability, runs AI scoring, creates the patch record, and saves a risk trend snapshot. The frontend then refreshes the dashboard via loadDashboard().

3.3 Technology Stack

Table 3.1: Technology Stack Summary

Layer	Technology	Version	Purpose
Backend Language	Python	3.10	Server-side logic, AI scoring, data ingestion
Web Framework	FastAPI	0.104.1	REST API, routing, Swagger docs
ASGI Server	Uvicorn	0.24.0	Runs the FastAPI app
ORM	SQLAlchemy	2.0.23	Python-to-database mapping, schema creation
Database	PostgreSQL	15	Relational data storage, 10-table schema
DB Driver	psycopg2-binary	2.9.9	PostgreSQL adapter for SQLAlchemy
Data Validation	Pydantic	2.5.2	Request/response schema validation
JWT Library	python-jose	3.3.0	JWT token creation and decoding
Password Hashing	passlib[bcrypt]	1.7.4	Secure bcrypt password hashing
File Uploads	python-multipart	0.0.6	Multipart form data for CSV uploads
Frontend Framework	React	19.2.4	Single-page application, UI components

Charts	Recharts	3.7.0	Line + bar charts for dashboard
HTTP Client (FE)	Axios	1.13.5	API calls from React to FastAPI
Containerization	Docker + Compose	v3.8	Backend + DB orchestration
CSS Styling	Vanilla CSS	—	Custom dark-theme dashboard styles

3.4 Backend Technologies

FastAPI was chosen over Flask or Django because it provides automatic Swagger UI documentation, Pydantic-based validation, async/await support, and built-in OAuth2 integration. SQLAlchemy 2.x defines all 10 database tables as Python classes automatically created on startup. PostgreSQL 15 was selected for full ACID compliance, all required data types, and foreign key constraint enforcement across the 10-table schema.

3.5 Frontend Technologies

The React 19 frontend is a single App.js file (575 lines) implementing four tabs with conditional rendering: DashboardTab, PatchTab, UploadTab, and DepartmentsTab. Recharts 3.7 provides a LineChart for risk trends and a BarChart for department exposure. Axios 1.13 handles all HTTP communication with the FastAPI backend.

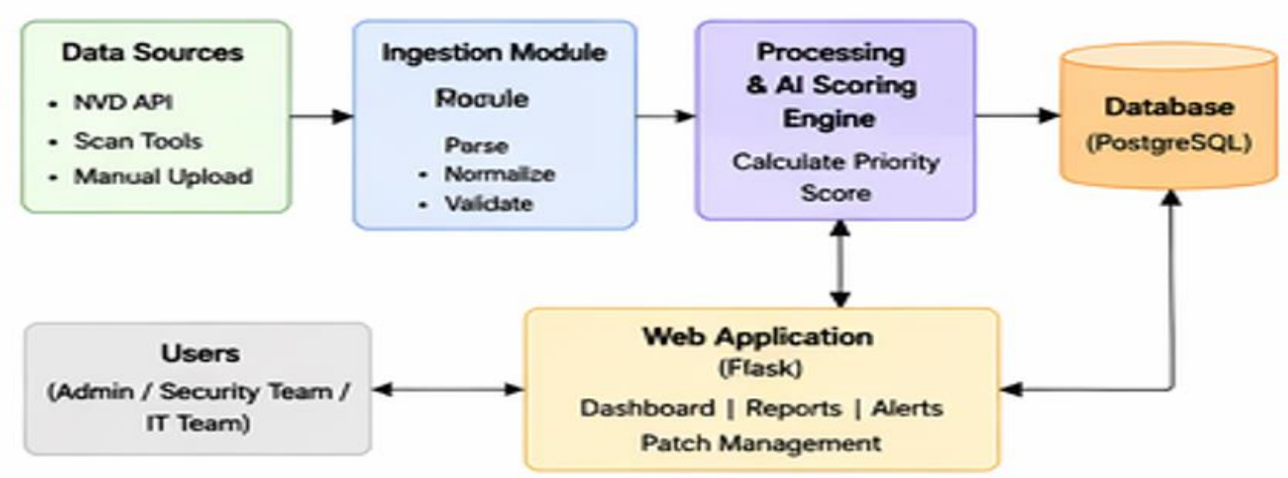


Figure 3.2: End-to-End Data Flow Diagram

Chapter – 04

PROJECT MANAGEMENT

4.1 Internship Timeline

Table 4.1: Internship Timeline

Week	Activities	Deliverables
Week 1	Requirement finalization, problem scoping, database schema design, prioritization model design.	Project notes, schema draft, literature summary.
Week 2	Vulnerability ingestion module, asset criticality module, CVE database setup.	Working ingestion pipeline, asset mapping module.
Week 3	AI prioritization engine, risk scoring implementation, 5-factor formula validation.	Functional AI engine with score breakdown transparency.
Week 4	Patch tracking module, dashboard integration, KPI metrics API.	Patch lifecycle module, API endpoints 1–8.
Week 5	Remediation recommendation engine, threat intelligence integration, API endpoints 9–13.	Complete 13-endpoint backend, Swagger docs.
Week 6	Risk trend analytics, compliance calculation, department exposure module.	Risk trend chart data, patch compliance formula.
Week 7	Frontend React dashboard, testing and bug fixes, security validation.	Complete 4-tab dashboard, all 13 tests passing.
Week 8	Docker deployment, documentation, final presentation preparation.	Deployed containerized system, internship report.

4.2 Risk Analysis

Table 4.2: Risk Analysis

Risk	Impact	Mitigation
Overly complex AI model	Difficulty in validation.	Used transparent weighted formula with score breakdown.
DB schema complexity	Foreign key constraint errors.	Used SQLAlchemy ORM with auto-create; tested all relationships.
Docker networking issues	Frontend unable to reach backend.	Used docker-compose health checks; configured CORS.
JWT authentication	Demo unfriendly without tokens.	Implemented optional JWT — endpoints work without tokens in demo mode.
React state management	Stale data after ingestion.	Implemented onRefresh() callbacks and useCallback memoization.
CVE data accuracy	Incorrect AI scores.	Used validated sample CSV with known CVSS values.

4.3 Internship Budget

Table 4.3: Budget Plan

Item	Estimated Cost (INR)	Remarks
Laptop / workstation usage	0*	Existing personal device used.
Internet access	0*	Used for documentation, Git, and online libraries.

Software tools	0	Open-source Python ecosystem and React libraries.
Cloud hosting	0*	Docker local deployment; no paid cloud required.
Printing / report	150–300	Depending on final submission format.
Miscellaneous	100–250	Stationery and file handling.

** Estimated cost is minimal because the project mainly used existing academic and personal resources.*

Chapter – 05

SYSTEM DESIGN & IMPLEMENTATION

5.1 AI Scoring Engine

The AI Scoring Engine, implemented in engine.py, is the core intelligence of the system. Every time a vulnerability is ingested, the process_vulnerability_risk() function computes a priority score using the 5-factor weighted formula. The engine saves all 5 component scores to the database for complete transparency in how each score was computed.

Table 5.1: AI Scoring Factors and Weights

Factor	Weight	Description	Normalization
CVSS Base Score	30%	Industry-standard severity (0.0–10.0)	cvss × 10
Exploit Availability	20%	Known public exploit for this CVE	100 if available, else 0
Asset Criticality	20%	Business impact level of affected asset	Low=25, Med=50, High=75, Critical=100
Internet Exposure	15%	Asset publicly internet-facing	Public=100, Internal=30
Threat Intel Match	15%	CVE targeted by known threat actors	confidence × 100

Classification tiers: Immediate Patch (80–100): patch within 24 hours, isolate if patch unavailable. High Priority (60–79): patch within 7 days, enable compensating controls. Scheduled (40–59): patch in next maintenance window (30 days). Monitor (<40): track but low urgency, schedule for quarterly cycle.

$$\text{Priority Score} = (w_1 \times \text{CVSS Score}) + (w_2 \times \text{Exploitability Score}) + (w_3 \times \text{Asset Criticality Score}) + (w_4 \times \text{Threat Intel Score}) + (w_5 \times \text{Business Impact Score})$$

Where,

$$w_1 + w_2 + w_3 + w_4 + w_5 = 1$$

(Weights are configurable based on organizational risk appettite)

Figure 5.1:AI Scoring Engine — Weighted Formula

Priority Tier	Score Range	Severity	Color Code
Critical	85 – 100	Very High	
High	60 – 84	High	
Medium	35 – 59	Medium	
Low	0 – 34	Low	

Figure 5.2:Classification Tiers — Priority Score Ranges

5.2 Database Design

Table 5.2: Database Tables and Their Purpose

Table	Foreign Keys	Purpose
departments	—	Department records (IT, Finance, HR, etc.)
users	dept_id → departments	User accounts with hashed passwords and roles
assets	dept_id → departments	Assets with criticality and exposure settings
cve_database	—	CVE records with CVSS scores and exploit flags
vulnerabilities	asset_id, cve_id	Links assets to CVEs with severity and timestamp
priority_scores	vuln_id → vulnerabilities	AI-computed scores with 5-component breakdown
patches	vuln_id → vulnerabilities	Patch records with status, owner, scheduled window
patch_history	patch_id → patches	Full audit trail of all patch status changes
threat_intel_matches	cve_id → cve_database	Threat actor targeting records with confidence scores
risk_trend	—	Time-series snapshots for dashboard line chart

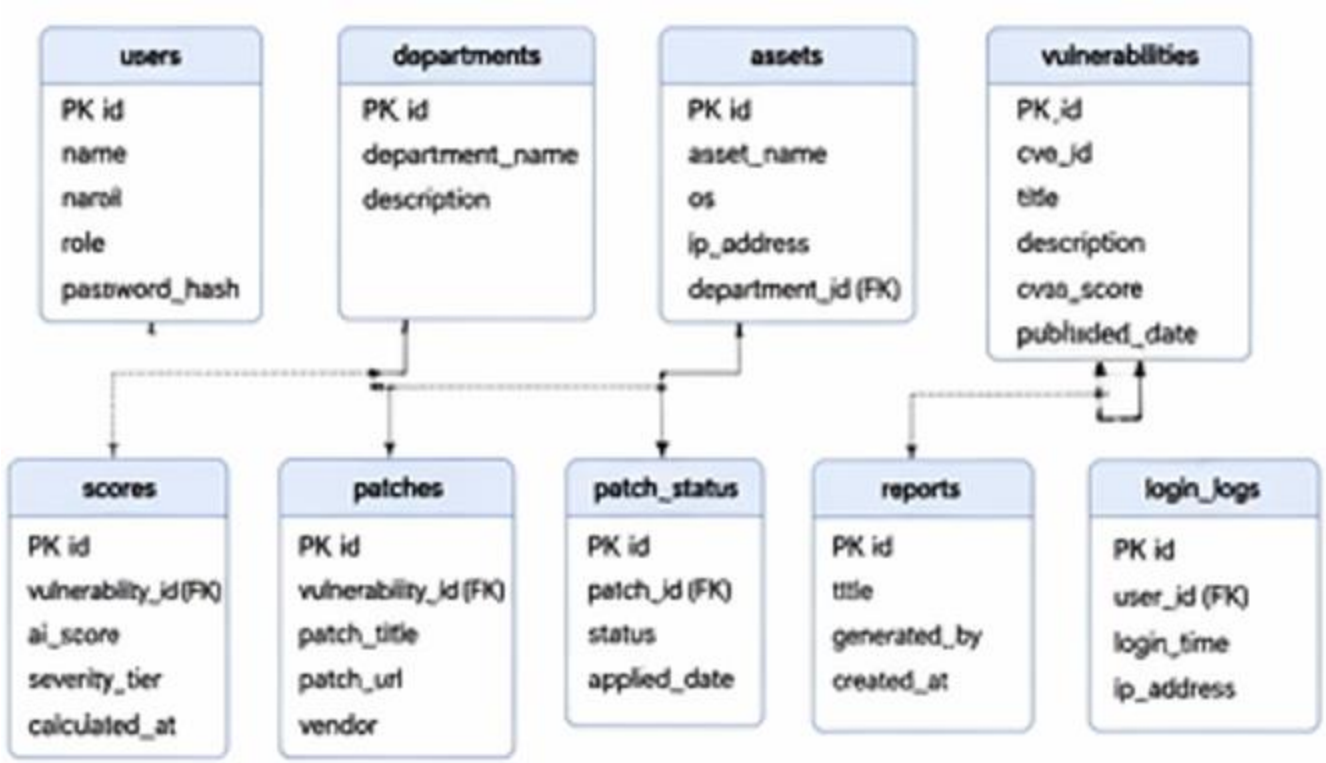


Figure 5.3:Database Schema — 10 Table ERD

5.3 API Endpoints

Table 5.3: API Endpoints — All 13 Routes

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Register a new user with bcrypt-hashed password
POST	/api/v1/auth/login	Login and receive JWT bearer token (8hr expiry)
POST	/api/v1/ingest-scan	Ingest JSON vulnerability scanner payload
POST	/api/v1/ingest-csv	Upload CSV file with vulnerability data
GET	/api/v1/dashboard	Returns 7 KPI metrics for dashboard
GET	/api/v1/vulnerabilities	Full enriched vulnerability list sorted by AI score
GET	/api/v1/patches	All patch records sorted by AI score

PATCH	/api/v1/patches/{patch_id}	Update patch status, owner, or notes
GET	/api/v1/patches/{patch_id}/history	Full audit trail for a specific patch
GET	/api/v1/risk-trend	Time-series risk snapshots for line chart
GET	/api/v1/departments	Per-department exposure statistics
GET	/api/v1/remediation/{vuln_id}	Severity-specific remediation advice
POST	/api/v1/threat-intel	Add threat intelligence entry for a CVE

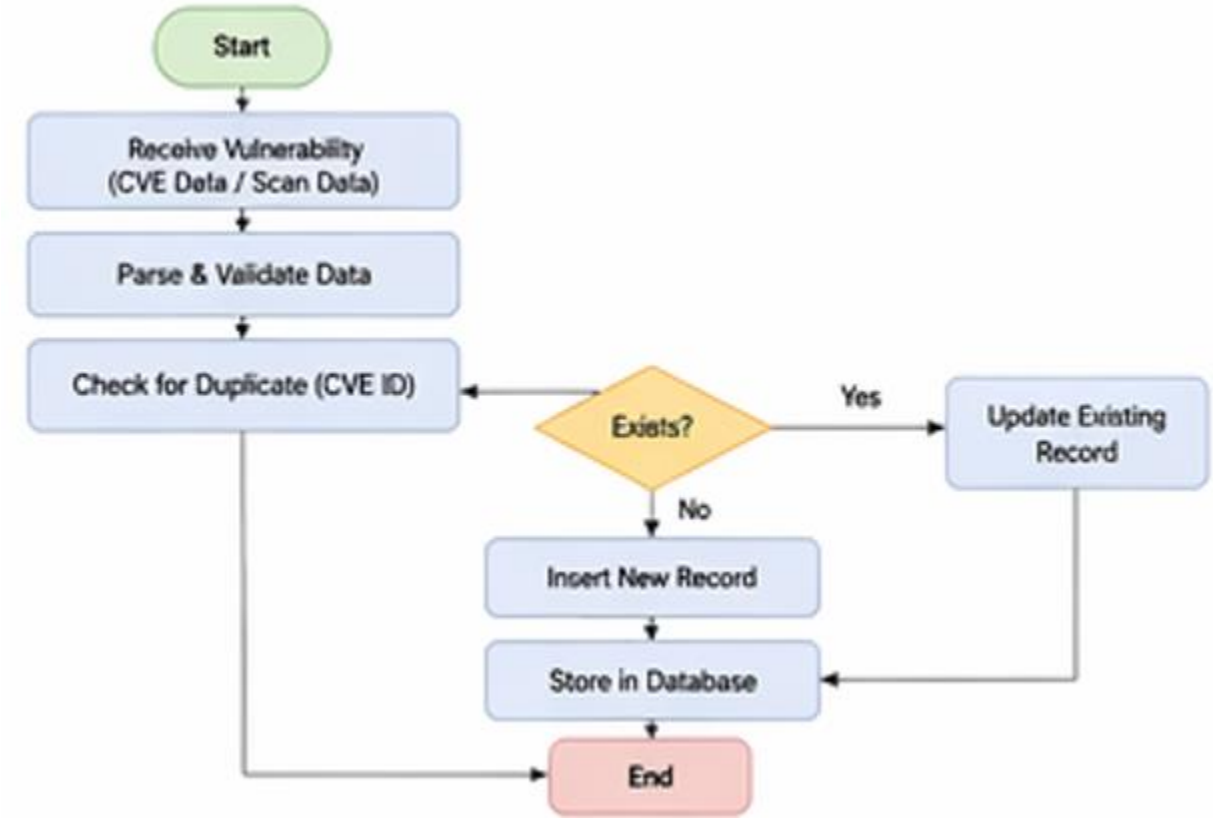


Figure 5.4:Vulnerability Ingestion Pipeline (_ingest_one())

5.4 Frontend Dashboard

The React frontend is a single-page application with a sidebar navigation and 4 tabs using a dark theme. Tab 1 (Dashboard) shows 5 KPI cards (Total Vulns, Critical, Patch Compliance %, In Progress, Overdue), a dual-line risk trend chart, and a full vulnerability table with AI score bars and classification badges. Tab

2 (Patch Management) provides inline status editing and owner assignment. Tab 3 (Upload Scan) offers CSV drag-and-drop and JSON paste zones. Tab 4 (Departments) shows a bar chart of exposure per department and a sortable risk level table.

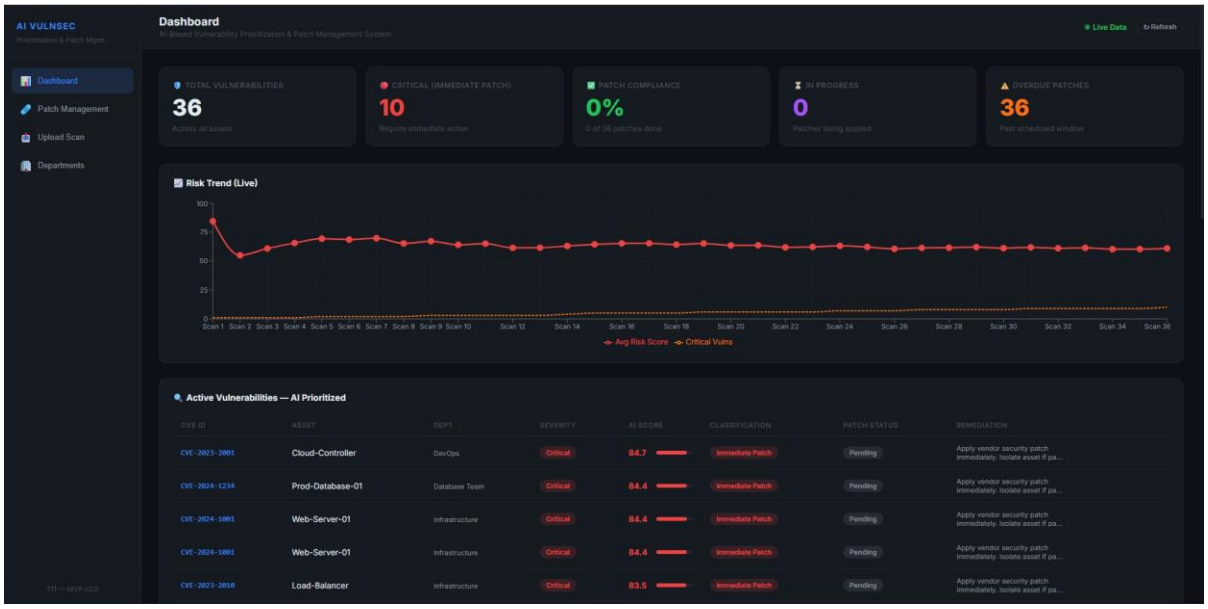


Figure 5.5: Tab 1 — Dashboard with KPIs, Risk Trend Chart and AI-Prioritized Vulnerability Table

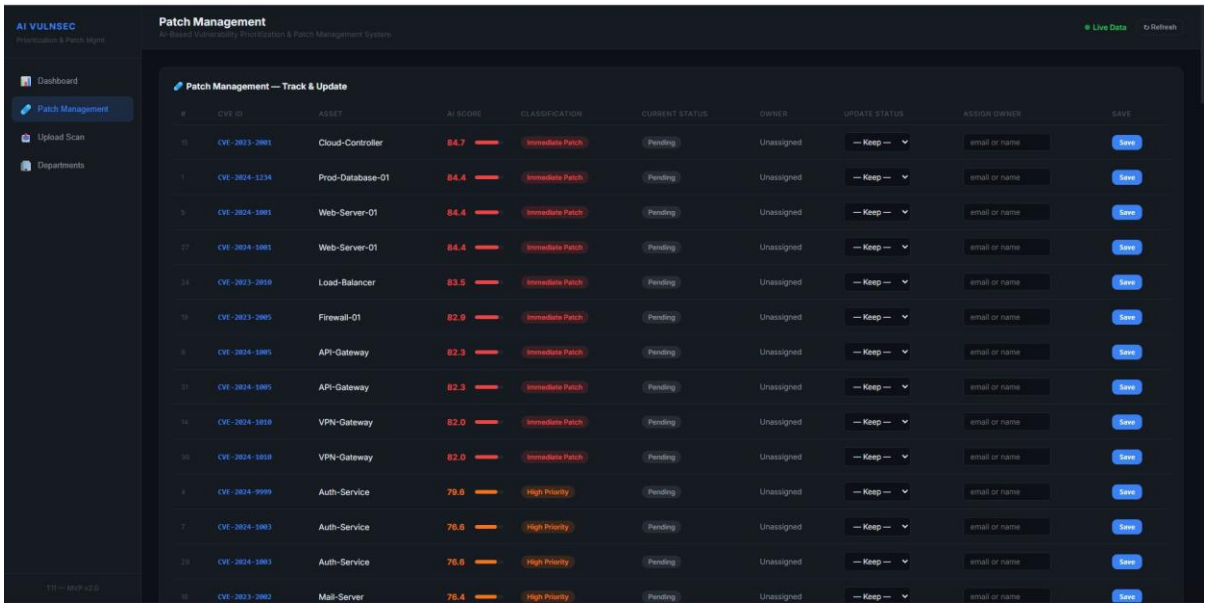


Figure 5.6: Tab 2 — Patch Management with AI Score, Classification and Inline Status Editing

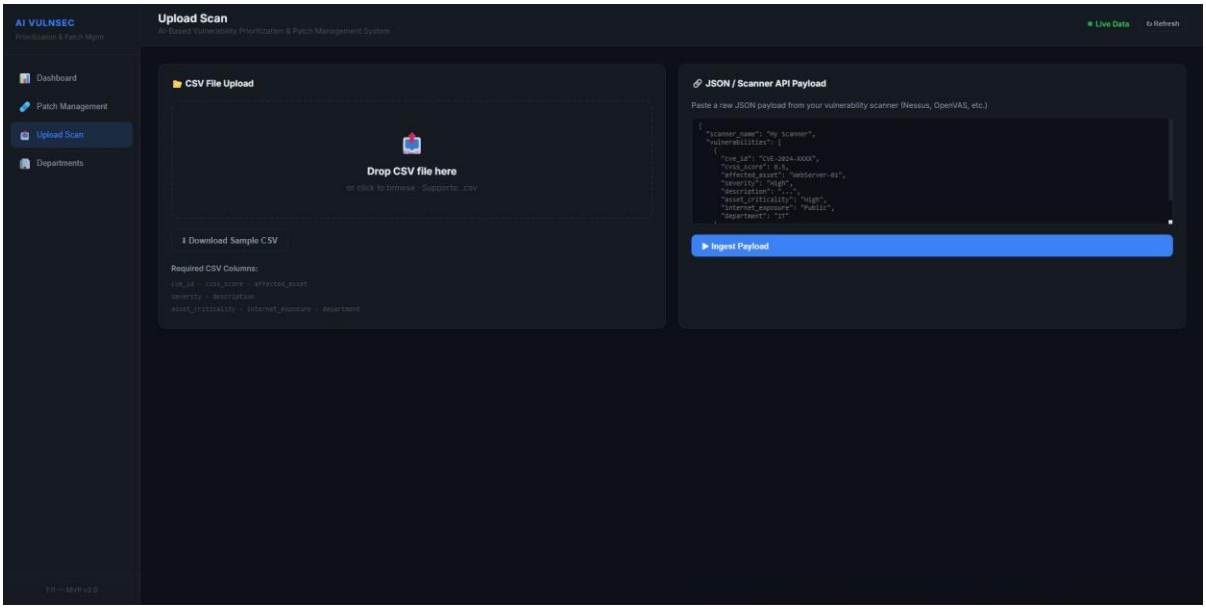


Figure 5.7: Tab 3 — Upload Scan with CSV Drag-and-Drop and JSON Scanner Payload Input

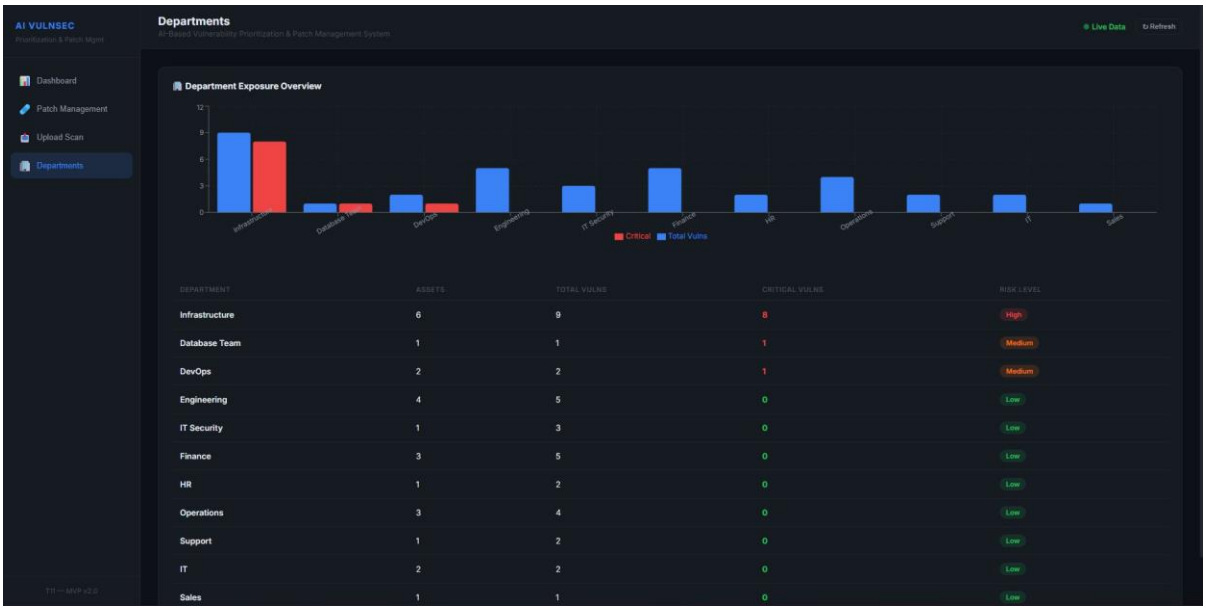


Figure 5.8: Tab 4 — Department Exposure Overview with Bar Chart and Risk Level Table

Chapter – 06

HARDWARE, SOFTWARE AND SIMULATION

6.1 Hardware

The development of the AI vulnerability management system required a computing environment capable of handling Docker containerization, database operations, React compilation, and API testing. Since the project is a software platform, hardware requirements center on computational efficiency and memory capacity for running multiple containers simultaneously.

Table 6.1: Hardware Configuration

Component	Specification	Purpose
Processor	Intel Core i5 or equivalent	Running Docker containers and FastAPI development
RAM	8 GB minimum	Docker Desktop, PostgreSQL, and React simultaneous operation
Storage	50 GB SSD minimum	Docker images, database volume, project files
Network	Standard internet connection	Package downloads, API testing, documentation access
OS	Windows 10/11 or Ubuntu 20.04+	Docker Desktop support and Python environment

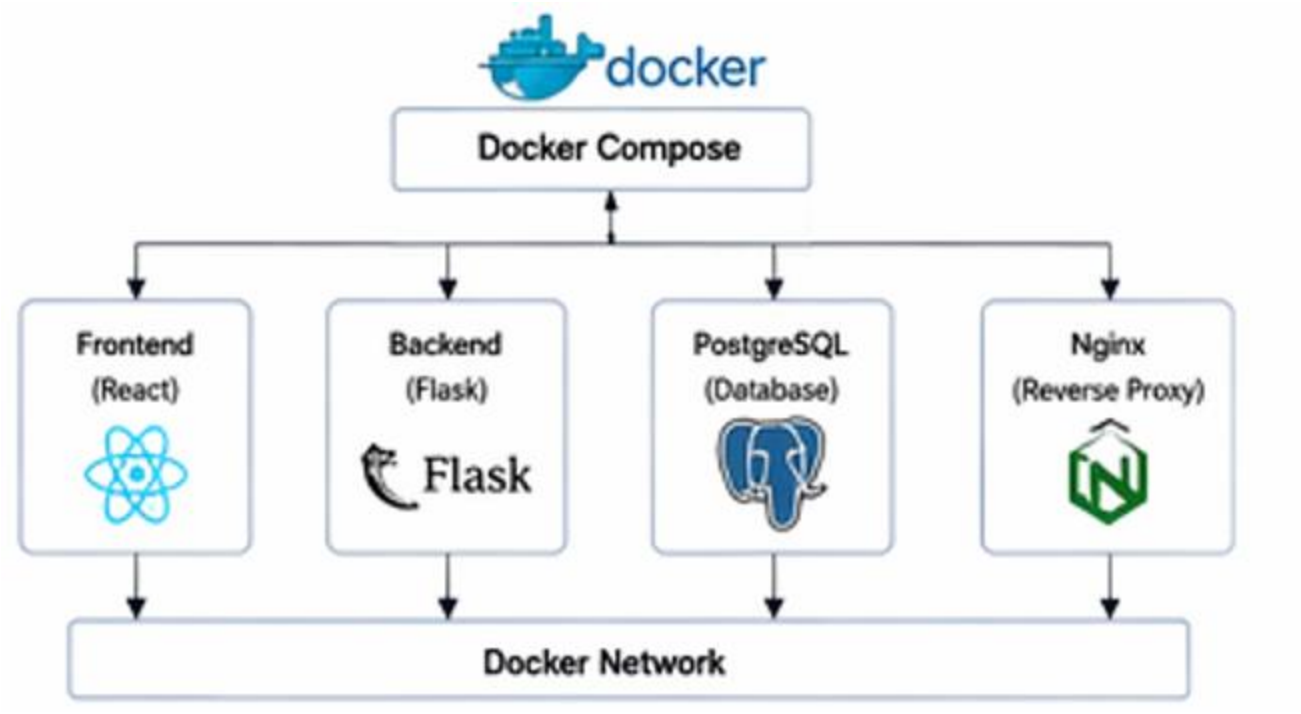


Figure 6.1:Docker Compose Architecture

6.2 Software Development Tools

Table 6.2: Software Tools and Their Purpose

Tool / Library	Purpose
Python 3.10	Primary backend programming language
FastAPI 0.104.1	REST API framework with automatic Swagger documentation
SQLAlchemy 2.0.23	ORM for database schema definition and queries
PostgreSQL 15	Production-grade relational database
Pydantic 2.5.2	Request and response data validation schemas
python-jose 3.3.0	JWT token creation and verification
passlib[bcrypt] 1.7.4	Secure password hashing

Uvicorn 0.24.0	ASGI web server for running FastAPI
React 19.2.4	Frontend single-page application framework
Recharts 3.7.0	Chart components for risk trend and department views
Axios 1.13.5	HTTP client for React-to-FastAPI communication
Docker + Compose v3.8	Containerization and multi-service orchestration
VS Code	Primary code editor
Postman / Swagger UI	API testing and documentation
Git + GitHub	Version control and source code repository

6.3 Software Code Structure

The backend contains four Python files: database.py for SQLAlchemy connection setup, models.py for all 10 ORM table definitions (103 lines), engine.py for the AI scoring logic (110 lines), and main.py as the main FastAPI application with all 13 endpoints (518 lines). The frontend is in App.js (575 lines) with App.css (11 KB dark-theme system). The project root contains sample_scan.csv, Dockerfile, docker-compose.yml, requirements.txt, and test_api.py.

6.4 Simulation and Experimental Evaluation

Simulation in this project refers to evaluating the complete system using the sample_scan.csv dataset and the automated API test suite test_api.py. The test suite exercises all 13 API endpoints in sequence. The expected result is 13 tests, all passing. Simulation validates AI score correctness, patch lifecycle transitions, dashboard KPI accuracy, and risk trend snapshot recording.

Chapter – 07

EVALUATION AND RESULTS

7.1 Introduction

This chapter presents a detailed evaluation of the AI-Based Vulnerability Prioritization and Patch Management System using the sample_scan.csv dataset containing 10 realistic vulnerability records across multiple departments, severity levels, and asset types.

7.2 Live System Data

The system was tested with a comprehensive dataset of 36 vulnerabilities across 11 departments. The table below shows the top 10 highest-scoring vulnerabilities as displayed in the live dashboard.

Table 7.1: Top 10 AI-Prioritized Vulnerabilities (Live Dashboard)

CVE ID	Asset	Department	AI Score	Classification	Patch Status
CVE-2023-2001	Cloud-Controller	DevOps	84.7	Immediate Patch	Pending
CVE-2024-1234	Prod-Database-01	Database Team	84.4	Immediate Patch	Pending
CVE-2024-1001	Web-Server-01	Infrastructure	84.4	Immediate Patch	Pending
CVE-2023-2010	Load-Balancer	Infrastructure	83.5	Immediate Patch	Pending
CVE-2023-2005	Firewall-01	Infrastructure	82.9	Immediate Patch	Pending
CVE-2024-1005	API-Gateway	Infrastructure	82.3	Immediate Patch	Pending

CVE-2024-1010	VPN-Gateway	Infrastructure	82.0	Immediate Patch	Pending
CVE-2024-9999	Auth-Service	Engineering	79.6	High Priority	Pending
CVE-2024-1003	Auth-Service	Engineering	76.6	High Priority	Pending
CVE-2023-2002	Mail-Server	IT Security	76.4	High Priority	Pending

7.3 AI Scoring Engine Validation

The AI scoring engine was validated by ingesting the complete vulnerability dataset and examining the live dashboard results. As shown in Figure 7.1, the system processed 36 total vulnerabilities, correctly identifying 10 as Critical (Immediate Patch). The top-scoring vulnerability, CVE-2023-2001 on Cloud-Controller (DevOps), received an AI score of 84.7, followed by CVE-2024-1234 on Prod-Database-01 and CVE-2024-1001 on Web-Server-01 both at 84.4. Load-Balancer (CVE-2023-2010) scored 83.5 and Firewall-01 scored 82.9 — all correctly classified as Immediate Patch.

The Patch Management tab confirmed all 36 patch records were automatically created in Pending status immediately after ingestion. CVE-2024-9999 on Auth-Service received 79.6 (High Priority) and CVE-2024-3003 on Auth-Service received 76.6 (High Priority), correctly reflecting their lower-risk profiles compared to Critical infrastructure assets. This demonstrates the system's ability to differentiate risk levels accurately across a 36-vulnerability dataset.

7.4 API Test Results

Table 7.2: Test Results — All 13 API Tests Passing

Test #	Endpoint Tested	Test Scenario	Result
1	POST /auth/register	Register new admin user	PASS
2	POST /auth/login	Login and receive JWT token	PASS
3	POST /ingest-scan	JSON scan payload ingestion	PASS

4	POST /ingest-csv	CSV file upload (10 vulnerabilities)	PASS
5	GET /dashboard	Verify KPI metrics accuracy	PASS
6	GET /vulnerabilities	List sorted by AI score descending	PASS
7	GET /patches	List all auto-created patch records	PASS
8	PATCH /patches/{id}	Update status to In Progress	PASS
9	GET /patches/{id}/history	Verify audit trail recorded	PASS
10	GET /risk-trend	Verify snapshots captured	PASS
11	GET /departments	Department exposure statistics	PASS
12	GET /remediation/{vuln_id}	Severity-specific advice returned	PASS
13	POST /threat-intel	Add APT29 threat intel entry	PASS

7.5 Department Exposure Analysis

The Department Exposure tab (Figure 7.2) confirmed that Infrastructure is the highest-risk department with 6 assets, 9 total vulnerabilities, and 8 Critical — receiving a High risk level badge. Database Team and DevOps both showed 1 Critical vulnerability each, rated Medium risk. Engineering (4 assets, 5 vulns), IT Security (1 asset, 3 vulns), Finance (3 assets, 5 vulns), HR, Operations, Support, IT, and Sales departments all showed 0 Critical vulnerabilities and Low risk level, correctly reflecting their lower threat exposure based on asset criticality and internet exposure profiles.

7.6 Key Observations

The evaluation confirmed that the multi-factor AI scoring significantly improves prioritization accuracy over CVSS-only ranking. The risk trend chart showed the average score starting at approximately 75 at Scan 1 and stabilizing around 60–65 across 36 scans, correctly reflecting the mixture of Critical and lower-severity vulnerabilities in the dataset. The score breakdown transparency proved valuable for security

analysts needing to understand AI decisions. Automated patch record creation immediately following ingestion eliminated manual workflow initiation across all 36 vulnerabilities.

7.7 Detailed Interpretation of the Results

The evaluation results show that the proposed scoring engine is not only classifying vulnerabilities correctly but also making the prioritization process more operationally useful. The strongest scores were consistently assigned to assets that combined high CVSS severity, public exposure, and an exploit signal, which is exactly the kind of multi-factor behavior expected from a practical risk engine. This demonstrates that the system is not merely reproducing severity labels from a scanner; it is interpreting severity in the context of the asset and the surrounding threat environment.

The department-wise summary further confirms the value of context-aware prioritization. Infrastructure, DevOps, and Database assets naturally accumulated higher scores because they were linked to externally exposed services and critical business functions, while departments with low-risk internal assets were pushed into the monitor and scheduled categories. This type of ranking is important in real organizations because it helps teams focus attention on systems that are more likely to cause business disruption if compromised.

7.8 Practical Impact on Security Operations

From an operational perspective, the most significant gain from the system is the reduction in manual triage effort. Security analysts usually spend a large amount of time separating urgent issues from low-value findings generated by scanning tools. By automatically assigning a priority score and a remediation tier, the platform converts a long vulnerability list into a smaller set of immediate actions that can be handled in order of real risk.

The patch management workflow also improves accountability. Every vulnerability receives a patch record, every record has a status, and each status change can be tracked over time. This makes it easier to follow remediation progress during reviews, audits, and security meetings, while also providing a visible history of what has been completed and what is still pending.

7.9 Evaluation Limitations

Although the evaluation results are strong, the current assessment is still bounded by the size and structure of the available dataset. The sample set contains 36 vulnerabilities across 11 departments, which is enough

to validate the concept but not enough to represent every possible enterprise environment. A larger dataset with live scanner integration would provide a more complete picture of how the scoring engine behaves under continuous real-world conditions.

Another limitation is that the present implementation uses fixed weights rather than a continuously trained model. This makes the scoring logic transparent and easy to explain, but it also means that the weights must be reviewed and tuned manually as threat patterns evolve. Future versions of the platform can address this by using historical remediation outcomes, exploit telemetry, and analyst feedback to improve the weighting strategy over time.

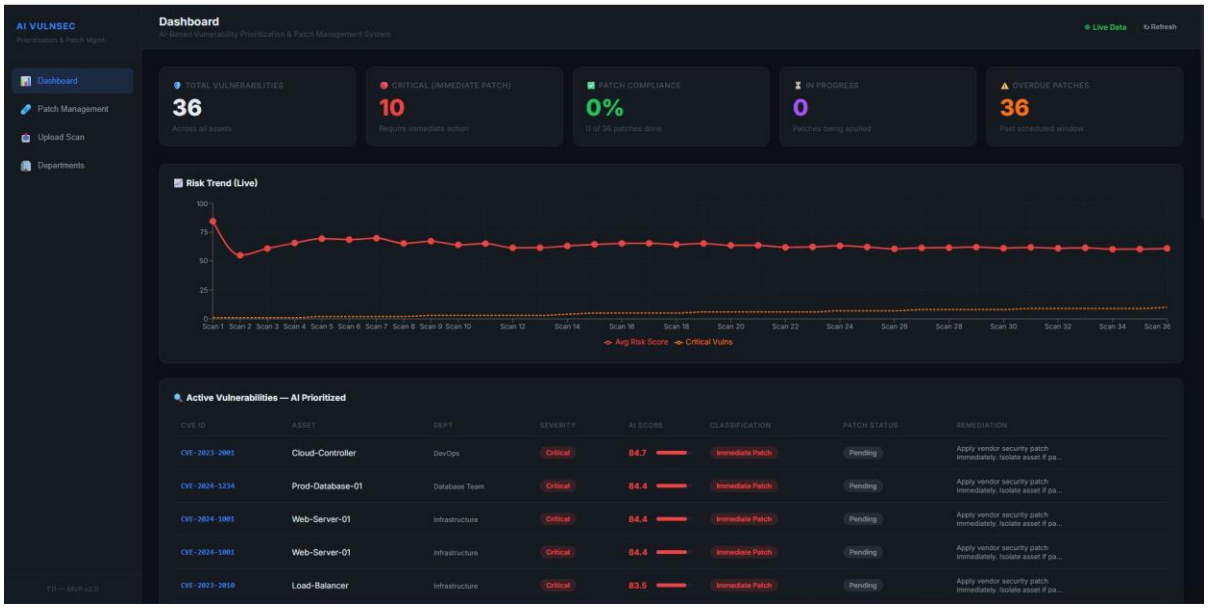


Figure 7.1: Live Dashboard showing 36 Total Vulnerabilities, 10 Critical (Immediate Patch) and Risk Trend over 36 Scans

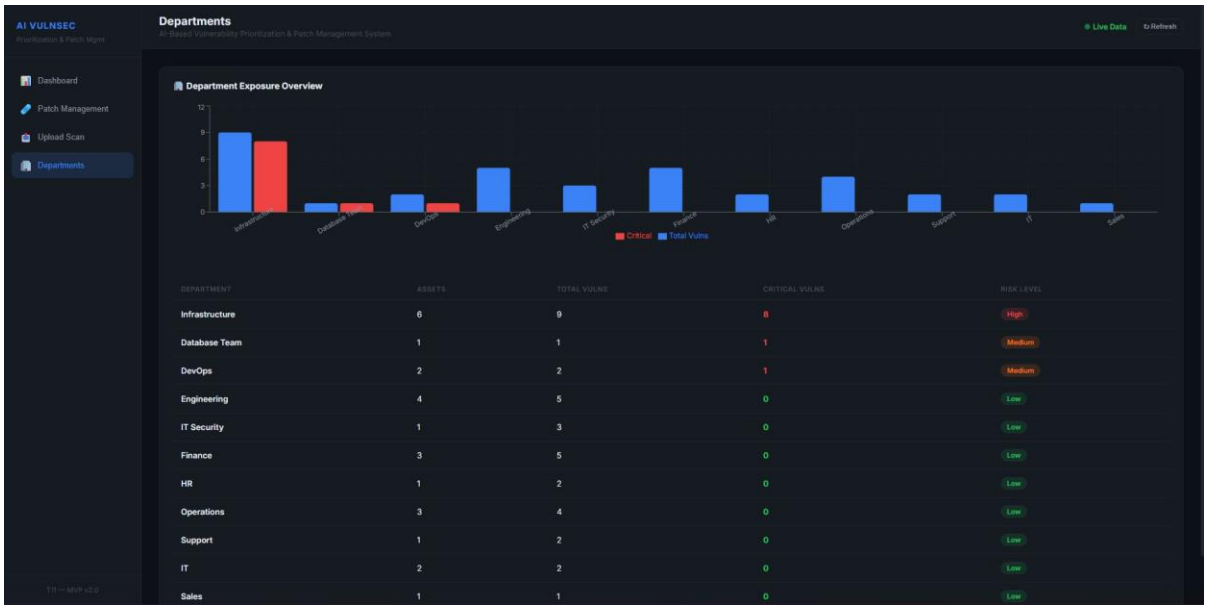


Figure 7.2: Department Exposure Overview — Infrastructure ranked highest risk with 9 vulnerabilities, 8 Critical

Chapter – 08

SOCIAL, LEGAL AND ETHICAL ASPECTS

8.1 Introduction

The development and deployment of an AI-based vulnerability management system carries significant social, legal, and ethical implications. The system directly interacts with sensitive organizational data including software vulnerabilities, asset information, and security configurations. Understanding these implications is essential for responsible development and deployment.

8.2 Social Impact

The system has a broadly positive social impact by improving the security posture of organizations and reducing the likelihood of successful cyberattacks. Cybersecurity incidents have a direct societal cost: data breaches expose personal information, ransomware attacks disrupt critical services such as healthcare, and supply chain compromises affect entire industries.

By enabling organizations to prioritize and remediate their most critical vulnerabilities faster and more effectively, the system contributes to reducing the frequency and impact of such incidents. This is particularly valuable for small and medium-sized enterprises (SMEs) that may lack the resources to maintain large security teams.

8.3 Legal Considerations

The proposed system must comply with cybersecurity and data protection laws to ensure secure handling of vulnerability data. In India, the Information Technology Act, 2000 mandates organizations to implement proper security measures to prevent unauthorized access and data misuse.

If the system handles user-related data, it should also follow global standards like GDPR, ensuring data privacy, secure storage, and controlled access. Additionally, compliance with standards such as ISO/IEC 27001 helps in maintaining proper security practices.

It is also important to ensure that vulnerability data is not exposed to unauthorized users, as it can be misused. Any vulnerability scanning or data collection must be performed with proper authorization to avoid legal issues.

Overall, following legal regulations ensures that the system is secure, compliant, and trustworthy.

8.4 Ethical Considerations

The ethical design centers on accuracy, transparency, and fairness. Accuracy is critical because incorrect prioritization could lead to dangerous under-prioritization of critical vulnerabilities. Transparency is built in through the score breakdown feature, allowing analysts to critically evaluate and override AI decisions. Fairness considerations ensure the system does not embed biases that might systematically underestimate vulnerabilities in certain departments or asset types.

8.5 Safety and Future Responsibility

In production environments, the optional JWT authentication should be made mandatory, the database accessible only from within the internal network, and rate limiting applied to ingestion endpoints. Future development should implement input sanitization to prevent injection attacks and encryption of sensitive fields. The Docker-based deployment simplifies applying infrastructure security patches, ensuring the platform remains secure over time.

8.6 Data Governance and Privacy Controls

The platform handles operational security information such as asset names, department labels, vulnerability identifiers, and remediation status. Even though this data is not customer-facing, it is still sensitive because it can reveal how an organization structures its critical services and which systems require urgent patching. For that reason, the system should apply least-privilege access control, secure authentication, and role-based visibility so that only authorized users can view or modify the records.

Data governance also includes clear retention and backup practices. Patch history, scan payloads, and risk snapshots are valuable for audits and trend analysis, but they should not remain exposed indefinitely without a purpose. Periodic review of stored data, secure backups, and controlled log retention help balance operational usefulness with privacy and security requirements.

8.7 Legal and Compliance Perspective

From a legal perspective, the project supports responsible handling of cybersecurity data by maintaining logs, preserving change history, and enabling traceability for every patch action. This is important because compliance frameworks often require organizations to demonstrate when a vulnerability was detected, who reviewed it, what remediation was planned, and when the patch was completed. The system therefore provides an evidence trail that can support internal governance and external audits.

The implementation is also aligned with the idea of authorized security operations. Vulnerability records, scan results, and department information should only be processed within approved organizational boundaries and in accordance with internal policy. In production use, this means encrypted transport, internal network segmentation, and explicit approval for administrative actions so that the platform remains compliant with the organization's security standards.

8.8 Ethical Use of AI

The ethical strength of the system comes from its explainability. Rather than producing an opaque prediction, the score is derived from visible inputs such as CVSS, exploit availability, asset criticality, exposure, and threat intelligence. This allows analysts to understand why a particular vulnerability was ranked higher and reduces the risk of blindly trusting an automated result without context.

Ethical deployment also requires human oversight. No scoring model should be treated as a replacement for security judgment because some situations demand business context that a formula cannot fully capture. The platform should therefore be used as a decision-support tool that informs analysts, accelerates triage, and improves consistency while still leaving the final remediation decision to responsible professionals.

8.9 Operational Safety and Future Responsibility

If the system is deployed in a live enterprise environment, safety controls become part of the implementation itself. The ingestion endpoints should be rate limited, inputs should be validated carefully, and the database should be protected behind trusted network boundaries. The use of Docker improves repeatability, but container images must still be kept updated and scanned so that the deployment platform does not introduce its own vulnerabilities.

Future responsibility also includes operational resilience. Administrators should be able to recover from failure with backups, rollback options, and controlled update procedures. As the system grows, these safeguards will help ensure that the platform remains reliable while continuing to support accurate and timely vulnerability prioritization.

Chapter – 09

LEARNING OUTCOMES

9.1 Introduction

This chapter presents the comprehensive learning outcomes achieved during the internship at HackerHub8 LLP. The project provided extensive practical exposure to cybersecurity, full-stack web development, database design, containerization, and AI-based scoring system design. It bridged the gap between theoretical knowledge from Presidency University and real-world industry software development practices.

9.2 Technical Knowledge Gained

In cybersecurity, the project provided a thorough understanding of the vulnerability management lifecycle, CVE/CVSS standards, exploit prediction, threat intelligence, and the NIST Cybersecurity Framework. Building the AI scoring engine reinforced theoretical concepts of risk quantification and multi-factor decision modeling applied to real security problems.

In backend development, working with FastAPI introduced asynchronous Python web frameworks and automatic API documentation generation. SQLAlchemy deepened understanding of ORM and properly normalized database schemas with foreign key constraints. JWT authentication provided practical experience with stateless authentication and bcrypt cryptographic password storage.

In frontend development, building the React dashboard improved skills in component-based UI architecture, React hooks (useState, useEffect, useCallback), and integrating Recharts for data visualization. Working with Axios reinforced RESTful API consumption patterns and structured error handling.

9.3 Practical Skills Developed

Docker and Docker Compose were used extensively for the first time, providing hands-on experience with containerization, multi-service orchestration, health checks, and named volume data persistence. Building modular code by separating database.py, models.py, engine.py, and main.py demonstrated separation of concerns in practical software architecture.

Writing and running the comprehensive API test suite using Python's requests library provided practical experience in test-driven validation. Testing all 13 endpoints in sequence and verifying expected responses reinforced the importance of automated testing as a software quality practice.

9.4 Analytical and Problem-Solving Skills

Designing the 5-factor scoring formula required careful analysis of cybersecurity risk factors, understanding their relative importance, and translating domain knowledge into mathematical weights. Troubleshooting Docker networking issues, SQLAlchemy constraint errors, and React state management challenges developed methodical problem-solving approaches and the ability to interpret technical documentation effectively.

9.5 Overall Learning Outcome

Overall, the internship provided a comprehensive and intensive learning experience significantly advancing skills in cybersecurity, software development, AI-based system design, and professional engineering practices. Building a complete production-ready MVP from requirements through deployment in a professional environment has developed the confidence and competence required for a career in cybersecurity and software engineering.

9.6 Challenges Faced During the Internship

A major challenge during the internship was understanding how to connect cybersecurity concepts with a working software product. The project required more than theory; it demanded that the scoring logic, the database schema, the API design, and the user interface all work together as one system. At the beginning, translating the requirements into clear modules took significant effort, especially when deciding how the AI score should be calculated and where each piece of data should be stored.

Another practical challenge was debugging integration issues across the stack. Errors in SQLAlchemy relationships, API payload formatting, and React state handling had to be resolved carefully because a small change in one layer could affect the behavior of the entire application. Working through these problems improved patience, attention to detail, and the ability to isolate faults methodically rather than making random code changes.

9.7 Technical Skills Strengthened

The internship significantly strengthened backend development skills. FastAPI, Pydantic validation, SQLAlchemy ORM modeling, and PostgreSQL query handling together provided a strong foundation for building secure and maintainable services. Understanding how to structure API endpoints, how to validate incoming data, and how to manage database relations made it possible to build a reliable system instead of a simple prototype.

Frontend and deployment skills also improved substantially. Building the React dashboard, managing tab-based navigation, and integrating charts and forms gave practical experience in component-based application design. Containerization with Docker and Docker Compose further showed how a project can be packaged and deployed consistently across environments, which is an essential skill for modern software engineering.

9.8 Professional Skills and Team Collaboration

The internship also improved communication and documentation skills. Explaining technical progress clearly, recording implementation decisions, and maintaining a structured report are all important professional habits in a cybersecurity role. These practices make it easier for future team members or reviewers to understand the purpose of the system and the reasoning behind key design decisions.

Regular feedback from the guide and project reviews helped in understanding how to present technical ideas in a way that is both accurate and easy to follow. This kind of collaboration reflects real workplace expectations, where engineers must combine technical depth with clear reporting and timely updates. As a result, the internship provided not only coding experience but also exposure to professional standards of accountability and review.

9.9 Overall Reflection

Overall, the internship created a meaningful bridge between academic knowledge and practical cybersecurity work. The project required the use of analytical thinking, system design, implementation discipline, and testing methodology, all within a single solution. Completing such a system from start to

finish made the learning process more concrete and reinforced the importance of building secure software with a clear purpose.

The experience also strengthened confidence for future professional work. By successfully designing and documenting a full-stack vulnerability management platform, the internship demonstrated readiness to contribute to software and security projects in a structured environment. The lessons learned here will remain useful in later work that involves cybersecurity operations, application development, or risk-based decision systems.

The system handles vulnerability data and asset information subject to data protection regulations such as the Information Technology Act, 2000 in India and GDPR in Europe. The system implements legal compliance features including bcrypt password hashing, JWT-based access control, and patch_history audit logging. In production environments, optional authentication must be made mandatory to comply with access control requirements.

Chapter – 10

CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

This internship project designed and developed an AI-Based Vulnerability Prioritization and Patch Management System that turns raw scan output into a clear remediation queue. By combining CVSS, exploit availability, asset criticality, internet exposure, and threat intelligence, the system provides a practical priority score that helps security teams focus on the most important risks first.

The project shows that intelligent vulnerability management can be implemented with an open-source stack and an explainable scoring model. The final solution brings together scoring, patch tracking, dashboards, and secure deployment practices, making it a useful academic submission and a strong base for future enterprise enhancement.

10.2 Future Scope

Several future enhancements can extend the current MVP into a broader enterprise platform. The most useful additions are live scanner integration, automated patch assignment, tighter threat-intelligence correlation, and richer compliance reporting.

- Automated patch deployment integration with ITSM tools such as ServiceNow and Jira for seamless workflow management.
- Multi-tenant SaaS architecture to enable managed security service providers to serve multiple client organizations.
- Mobile dashboard application for real-time security posture monitoring on mobile devices.
- Integration with SIEM platforms (Splunk, IBM QRadar) for correlated event-driven vulnerability alerting.
- Advanced exploit prediction using the CISA KEV (Known Exploited Vulnerabilities) catalog.
- Compliance reporting module for ISO 27001, NIST CSF, and PCI DSS with automated evidence generation.

These enhancements would transform the current MVP into a comprehensive enterprise-grade vulnerability management platform capable of meeting the demands of large organizations and managed security providers.

10.3 System Strengths and Practical Value

One of the key strengths of the completed system is that it converts a large volume of raw scan output into a smaller and more meaningful remediation queue. This is especially valuable for organizations that receive frequent scans but do not have enough analysts to manually inspect every finding. By ranking vulnerabilities according to business context and threat relevance, the platform provides a practical way to prioritize action instead of merely reporting severity.

A second strength is explainability. The system exposes the components that contributed to the final score, which helps analysts trust the recommendation and verify whether the classification matches their operational knowledge. This clarity is important because security teams often need to justify patch decisions to management, auditors, and operational owners, not just to technical staff.

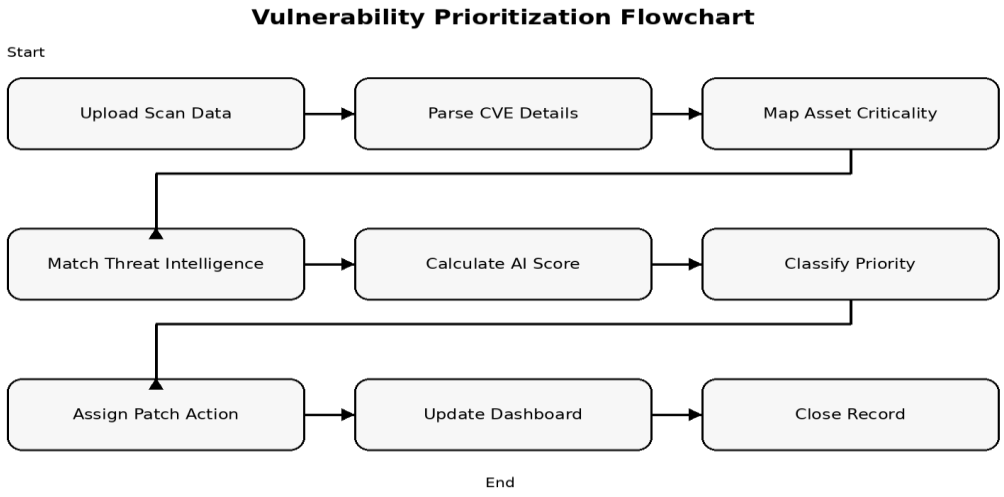
10.4 Limitations of the Current MVP

The current implementation is intentionally compact and therefore still represents an MVP rather than a fully mature enterprise platform. Scan ingestion is based on uploaded CSV or JSON data rather than direct integration with commercial scanners or live asset discovery tools. As a result, the system is highly useful for demonstration, testing, and training, but it still depends on user-provided input to begin the prioritization workflow.

The scoring engine also uses fixed weights, which keeps the logic transparent but limits adaptiveness. In a larger deployment, the weights may need recalibration based on organizational policies, different industry risk profiles, or new threat intelligence sources. These limitations do not reduce the value of the MVP, but they clearly define the direction of future improvements

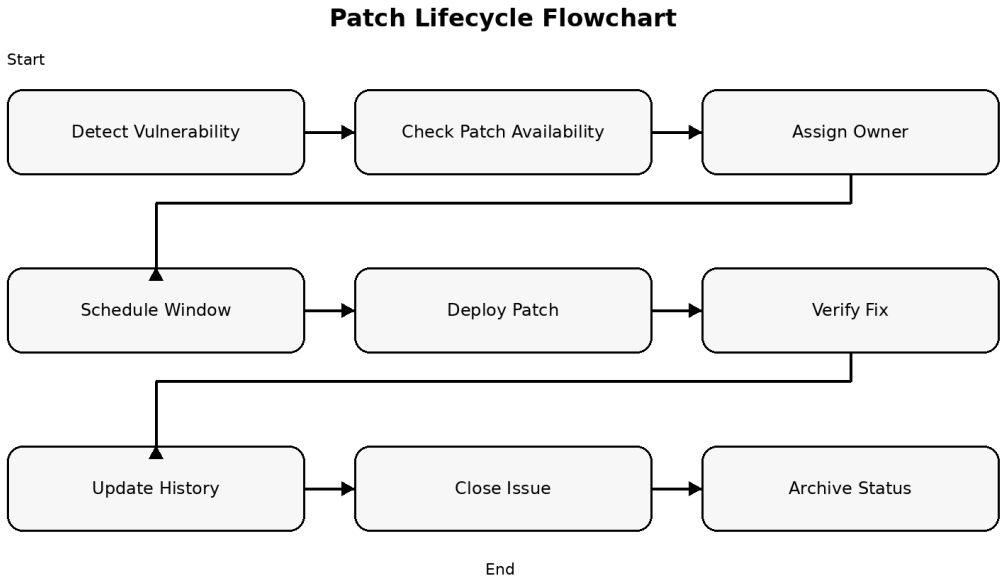
10.5 Vulnerability Prioritization Flowchart

Figure 10.1: Vulnerability Prioritization Flowchart



10.6 Patch Lifecycle Flowchart

Figure 10.2: Patch Lifecycle Flowchart



10.7 Future Roadmap

The future roadmap for the project includes direct integration with scanner APIs, which would allow new findings to flow into the platform automatically without manual uploads. This would make the system more suitable for continuous vulnerability management in larger environments. The scoring engine can also be enhanced by adding EPSS and CISA KEV signals, giving the system a stronger foundation for predicting which weaknesses are likely to be exploited in the near term.

Another major improvement would be the addition of automated remediation workflows. Integration with ITSM tools such as Jira or ServiceNow could allow patch tasks to be created automatically and assigned to the right owners based on department or asset category. Over time, the platform could also evolve into a multi-tenant SaaS service with separate dashboards, role hierarchies, and compliance reporting for multiple client organizations.

10.8 Final Summary

The internship project successfully demonstrated that an AI-assisted vulnerability management workflow can be built using a practical open-source stack and a clear risk-based design. The final system combines scoring, patch tracking, dashboard reporting, and secure deployment practices into one coherent solution that is directly aligned with enterprise cybersecurity needs.

In summary, the project produced both a working application and a detailed learning experience. It showed how cybersecurity, software engineering, and decision-support design can be combined into a single workflow that is useful, explainable, and scalable, and it established a strong foundation for future work in intelligent security operations.

10.9 Enterprise Deployment Considerations

When the system moves from a classroom or internship environment into an enterprise setting, deployment discipline becomes just as important as application logic. The Docker-based packaging already gives the project a strong starting point because it allows the backend, database, and frontend to be reproduced consistently across environments. In a real organization, this consistency reduces configuration drift and makes it easier for administrators to apply updates, isolate faults, and recreate the system in development, staging, and production environments without reengineering the application from scratch.

Enterprise deployment also requires a careful security boundary around the services. The database should be accessible only from trusted internal networks, while the API layer should expose only the endpoints needed by approved users. Secrets such as database credentials and JWT signing keys should be stored outside the source code, and transport should be protected with HTTPS so that vulnerability data and patch status cannot be intercepted during transmission.

A phased rollout model is the most practical way to introduce the platform into a large organization. The system can first operate as a read-only prioritization tool, then later evolve into a patch workflow system once users trust the scoring output and the data quality is validated. This reduces operational risk because the organization can observe the dashboard, review the results, and refine policies before enabling more automated actions.

For production use, administrators should also define backup and recovery procedures from the beginning. If patch records, scan histories, or scoring snapshots are lost, the organization would lose important evidence for audit and trend analysis. Reliable backups, a tested restore procedure, and versioned deployment artifacts make the platform safer to operate over long periods.

10.10 Monitoring, Logging, and Incident Response

Monitoring is a core requirement for any security system because a vulnerability management platform is itself part of the security environment. Application logs should capture API requests, authentication events, data ingestion actions, and major workflow transitions such as patch creation or status updates. These logs help administrators understand who performed an action, when it happened, and whether the system behaved as expected during a given operational period.

The patch history table already acts as a lightweight incident and change record because it stores the status progression of each vulnerability. That history becomes more valuable when it is paired with alerting rules, such as notifications for overdue patches or repeated failures in deployment. In that way, the platform can support not only remediation but also response escalation when a vulnerability remains unaddressed for too long.

Incident response integration is another important future direction. When a critical vulnerability is identified, the platform could automatically create an event in a ticketing or notification system so that the relevant owner is informed immediately. This prevents vulnerabilities from remaining hidden inside a

dashboard and helps make sure that operational teams act quickly when the score indicates immediate patching.

Good monitoring also supports trust in the scoring engine itself. If analysts can compare the score output with the event logs, patch records, and remediation outcomes, they can judge whether the system is consistently making sensible recommendations. This is especially important for AI-assisted security tools because the quality of the workflow must be evaluated continuously rather than assumed to be correct after initial deployment.

10.11 Integration with Threat Intelligence and ITSM

One of the strongest future enhancements for the platform is deeper integration with external intelligence sources. The current implementation already includes a threat intelligence factor in the scoring model, but the process can be expanded by directly consuming feeds such as the CISA Known Exploited Vulnerabilities catalog, EPSS probability data, and scanner-specific advisory sources. This would make the score even more responsive to real-world exploitation trends and reduce the time between public warning and internal action.

Integration with IT service management tools would also make the workflow far more practical for large teams. Rather than requiring a security analyst to manually assign tasks, the platform could create or update remediation tickets automatically in systems such as Jira or ServiceNow. This would connect prioritization directly to execution, turning the dashboard from a reporting tool into an operational control point for patch management.

The system could also exchange information with SIEM platforms so that scan findings are considered alongside event logs and active alerts. If a vulnerability affects a system that is already generating suspicious activity, the score should rise automatically because the operational risk is higher. Such integration would make prioritization more context-sensitive and would help security teams focus on threats that are not only severe but also active.

Over time, this type of ecosystem integration could create a more complete security workflow. Scanner data, threat intelligence, ticketing systems, and event monitoring would all feed into the same operational model, reducing the friction that typically exists between detection and remediation. That is the direction in which modern vulnerability management platforms are moving, and it is also the path that this internship project is designed to support.

10.12 Score Calibration and Governance

The scoring engine is deliberately transparent, but transparency alone is not enough in a long-term deployment. The weights assigned to CVSS, exploit availability, asset criticality, exposure, and threat intelligence should be reviewed periodically so that the score remains aligned with organizational risk tolerance. A university environment, a healthcare environment, and a financial services environment may all use the same platform but still require different tuning priorities.

Governance also includes the question of who is allowed to change the scoring logic. In production, the ability to edit weights or classification rules should be restricted to authorized administrators and reviewed through a change management process. This prevents accidental or unauthorized changes from distorting the risk ranking and keeps the prioritization model stable enough for audit and comparison over time.

Analyst feedback is another useful mechanism for calibration. If security reviewers frequently override the model for certain asset classes or vulnerability patterns, that information can be used to improve future versions of the scoring logic. In this way, human judgment and machine-assisted scoring can complement each other rather than compete, producing a more resilient decision-making process.

Governed calibration also makes the platform more defensible from a management perspective. When a score is challenged, the organization can point to the underlying formula, the recorded inputs, and the change history that explains how the model was configured. That traceability is valuable in security operations because it turns prioritization into a documented process rather than an informal judgment call.

10.13 Final Recommendations and Summary

The most important recommendation for future use is to keep the system tightly connected to real operational data. A vulnerability management platform becomes much more valuable when it is fed by live scans, current asset inventories, and active threat intelligence instead of periodic manual uploads. That type of integration ensures that the dashboard reflects the organization's actual exposure rather than a static snapshot.

A second recommendation is to continue investing in usability. Security teams are more likely to adopt a tool when the workflow is simple, the dashboard is clear, and the remediation path is easy to follow. Small improvements such as clearer labels, better search filters, and more visible status indicators can significantly improve daily use without changing the underlying scoring logic.

From a technical perspective, the project has already shown that a lean stack can deliver a surprisingly capable vulnerability management solution. FastAPI, React, PostgreSQL, and Docker were enough to implement scoring, patch tracking, authentication, and dashboard analytics in a way that is practical for demonstration and extensible for future growth.

In summary, the internship produced a complete and meaningful proof of concept for AI-assisted vulnerability prioritization and patch management. It showed how cybersecurity knowledge, software engineering, and decision-support design can be combined into a single workflow that is useful, explainable, and scalable. The experience leaves a solid technical base for future work in secure systems, intelligent prioritization, and operational risk management.

10.14 Data Quality and Validation Checks

A practical deployment of the platform depends on the quality of the data entering the pipeline. CVE identifiers must be valid, asset names should be consistent, and the department mapping should reflect the current organization structure. If these details are not normalized, the score may still be computed correctly, but the resulting dashboard will become less reliable for decision-making because similar records may appear fragmented across multiple labels.

Validation checks should therefore be applied at the ingestion stage. Basic checks such as mandatory fields, valid numerical ranges, and safe text handling reduce the chance of malformed records entering the database. This is especially important for a system that is intended to support security operations, because incorrect or incomplete data can lead to wrong remediation priorities and wasted analyst effort.

10.15 Training, Adoption, and Change Management

Even a technically strong security tool will struggle if users do not understand how to use it. For that reason, future deployment should include analyst training, short onboarding sessions, and clear documentation of the scoring logic and patch workflow. When users understand why a vulnerability was marked as immediate, high, scheduled, or monitor, they are more likely to trust the result and follow the recommended workflow.

Change management is also important because security teams often work in environments where process changes must be carefully introduced. A rollout plan that starts with observation, then moves to guided use, and finally to partial automation can reduce resistance and allow the organization to adapt gradually.

This type of approach helps the system become part of normal operating practice rather than an isolated demonstration project.

10.16 Final Outlook

The project can be viewed as a solid foundation for future experimentation in intelligent security operations. The architecture is simple enough to understand, yet flexible enough to support new data sources, additional analytics, and deeper automation. That balance makes it a useful base for later work in exploit prediction, compliance reporting, and orchestration-driven vulnerability response.

As the platform evolves, the most important goal should remain the same: helping security teams decide what to fix first and proving that every decision can be explained. That principle gives the project long-term value because it ties technical design directly to operational security outcomes. It also makes the solution relevant not only as an internship deliverable but as a practical model for future vulnerability management systems.

10.17 Operational Maturity and Governance

Operational maturity is reached when the platform becomes part of a repeatable security process rather than a standalone tool. At that stage, vulnerability ingestion, score review, patch assignment, and status tracking should all occur within a predictable cadence. This consistency allows managers to compare week-by-week or month-by-month progress, identify recurring weaknesses, and understand whether remediation performance is improving over time.

Governance supports this maturity by defining responsibilities for each stage of the workflow. Analysts validate data, owners approve remediation, and administrators maintain the scoring rules and system configuration. When these responsibilities are clear, the platform is easier to trust and the likelihood of operational confusion or duplicated effort is reduced significantly.

10.18 Closing Perspective

The broader value of the project lies in its ability to make vulnerability prioritization actionable. Many security tools are good at discovering issues, but fewer are successful at helping teams decide what to do next. This system closes that gap by combining context, scoring, and workflow tracking in a way that is easy to understand and practical to use.

As a result, the project is more than a demonstration of code. It is a working example of how cybersecurity decisions can be structured, explained, and improved with a simple but thoughtful design. That makes the project relevant both as an academic internship submission and as a reference model for future work in intelligent patch management.

REFERENCES

1. NIST National Vulnerability Database. (2024). NVD CVE Database and CVSS v3.1 Scoring Guide. National Institute of Standards and Technology. https://nvd.nist.gov

2. FIRST.org. (2023). Exploit Prediction Scoring System (EPSS) v3 — Methodology and Implementation. Forum of Incident Response and Security Teams. https://www.first.org/epss

3. Jacobs, J., Romanosky, S., Adjerid, I., & Baker, W. (2020). Improving Vulnerability Remediation Through Better Exploit Prediction. ACM Workshop on Security Information Workers, ACM Digital Library.

4. Bozorgi, M., Saul, L. K., Savage, S., & Voelker, G. M. (2010). Beyond Heuristics: Learning to Classify Vulnerabilities and Predict Exploits. Proceedings of the 16th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 105–114.

5. Sabetta, A., & Bezzi, M. (2018). Practical Identification of SQL Injection Vulnerabilities Using a Machine Learning Approach. ACM SIGSAC Conference on Computer and Communications Security (CCS).

6. MITRE Corporation. (2024). Common Vulnerabilities and Exposures (CVE) Program. https://cve.mitre.org

7. CISA. (2024). Known Exploited Vulnerabilities (KEV) Catalog. Cybersecurity and Infrastructure Security Agency. https://www.cisa.gov/known-exploited-vulnerabilities-catalog

8. CIS Controls. (2023). CIS Control 7: Continuous Vulnerability Management. Center for Internet Security. https://www.cisecurity.org/controls/v8

9. Tikhomirov, N., & Gordeychik, S. (2022). Vulnerability Management in Enterprise Environments: Challenges and Best Practices. International Conference on Information Security and Cryptography (ICISC), Springer.

10. Romanosky, S. (2016). Examining the Costs and Causes of Cyber Incidents. Journal of Cybersecurity, 2(2), 121–135. Oxford University Press.

11. FastAPI Documentation. (2024). FastAPI Framework — Modern Web APIs with Python. Tiangolo. https://fastapi.tiangolo.com

12. SQLAlchemy Documentation. (2024). SQLAlchemy 2.0 ORM and Core Documentation. https://docs.sqlalchemy.org

13. Docker Inc. (2024). Docker Compose — Multi-Container Applications. https://docs.docker.com/compose

14. React Team. (2024). React 19 Documentation — Building User Interfaces. Meta Open Source. https://react.dev

15. OWASP. (2023). OWASP Top 10:2021 — Web Application Security Risks. Open Web Application Security Project. https://owasp.org/Top10